

Developer Guide

Web-Based PAR Index System

Team TIMESWARE

CO2060 — Faculty of Engineering, University of Peradeniya

Abstract

This developer guide documents the architecture, technology stack, repository structure, local setup, authentication and authorization, core domain logic, PAR scoring paths, machine-learning lifecycle, training dataset pipeline, audit logging, testing, troubleshooting, and current deployment model of the Web-Based PAR Index System. The system consists of a React/Three.js frontend, Spring Boot backend, FastAPI machine-learning service, and MySQL database orchestrated with Docker Compose.

Index Terms

PAR Index, orthodontics, React, Three.js, Spring Boot, FastAPI, JWT, MySQL, Docker Compose, machine learning, audit logging.

I. ARCHITECTURE OVERVIEW

Four services orchestrated with Docker Compose: a React frontend (Three.js 3D viewer), a Spring Boot backend (REST API, JWT auth, RBAC), a FastAPI ML microservice (geometric landmark detection + a separately-trained PAR prediction model), and MySQL. The frontend never calls the ML service directly — all requests go through the backend, which also writes every significant action to the audit log.

II. TECHNOLOGY STACK

- Frontend: React, Three.js
- Backend: Spring Boot, Spring Security (JWT), JPA
- ML Service: FastAPI (Python) — geometric landmark detector (versioned, e.g. geometric-v1) + a separate trained PAR-prediction model (versioned, e.g. v1.0)
- Database: MySQL, schema owned entirely by Flyway migrations
- Infra: Docker Compose, Maven (committed wrapper — see mvnw/mvnw.cmd)

III. REPOSITORY STRUCTURE

```
code/
  frontend/    React app
  backend/     Spring Boot app
  ml_service/  FastAPI app
  database/    init.sql (schema owned by Flyway,
not this file)
  docker-compose.yml
docs/         All project documentation
```

IV. LOCAL SETUP

A. Frontend

```
cd code/frontend
npm install
npm run dev
```

B. Backend

```
cd code/backend
./mvnw spring-boot:run
```

Requires JAVA_HOME pointing at a full JDK 17 (not just a JRE).

C. ML Service

```
cd code/ml_service
pip install -r requirements.txt
python -m uvicorn app.main:app --reload
```

See docs/LandmarkDetectorFixHistory.md before modifying app/core/landmark_detector.py — it documents real axis-detection and coordinate-space bugs already found and fixed.

D. All Services Together (Docker Compose)

```
cd code
docker compose up --build
```

- Frontend: http://localhost:5173
- Backend: http://localhost:8081
- ML service: http://localhost:8000
- MySQL: localhost:3307 → container 3306

Environment variables go in a .env file at code/ (not committed) — see README.md's variable table.

V. AUTHENTICATION AND AUTHORIZATION

JWT-based. The signing secret is supplied via environment variable and must decode to at least 32 bytes — the application refuses to start otherwise (see JwtUtil). Roles are enforced server-side with Spring Security annotations, e.g. @PreAuthorize("hasRole('ORTHODONTIST')"), not just hidden in the UI.

Role	Created how	Access
ADMIN	Pre-seeded only	User management, read-only patient view, submissions overview, ML engine, audit log
ORTHODONTIST	Self-registered	Patients, cases, 3D upload, scoring, finalization, reviewing training submissions
UNDERGRADUATE	Self-registered	Submit training datasets, track own submissions

VI. CORE DOMAIN LOGIC

A. Patients and Cases

A patient's first case is always Pre-treatment. A Post-treatment case cannot be created until a Pre-treatment case for that patient is finalized — the frontend then auto-selects Post-treatment and links the new case to that finalized Pre-treatment case for outcome comparison.

B. 3D Model Upload

Every case requires exactly three files — Upper Arch, Lower Arch, Buccal View — STL or OBJ, max 50MB each, stored with path-traversal-protected storage (StorageService). Uploading all three automatically triggers an asynchronous ML landmark prediction and an ML PAR prediction in the background — the frontend polls or displays these once ready.

C. Three Independent PAR Scoring Paths

This is worth understanding precisely, since it's easy to conflate the three:

Path	How it's produced	score_source
Manual PAR Scoring	Clinician selects each of 7 British-Standard components against the on-screen rubric (see §7)	MANUAL
3D Landmark auto-score	Geometric detector places up to 31 landmarks/arch (optionally reviewed/corrected by the clinician); Auto-Calculate PAR computes a score from confirmed points	AUTO_LANDMARK
ML Predicted PAR	A separately trained model predicts PAR directly from the uploaded models — no landmark placement involved at all	ML

Note: These three can disagree significantly in practice — in one real walkthrough, the landmark auto-score was 39 and the ML prediction was 2.5 for the same case. This is documented, expected behavior given the ML model's current low training-data volume (see §8), not a bug to silently reconcile.

D. Finalization

Finalizing requires the user to type CONFIRM in a dialog — a deliberate friction point since finalizing locks the case permanently. An admin can un-finalize a case when genuinely needed (this action is itself audit-logged, e.g. "unfinalized by admin ... reason=...").

E. Outcome Classification (Post-treatment)

When a Post-treatment case is scored, the system compares its weighted PAR to the linked Pre-treatment score and classifies the result — e.g. Greatly Improved, Improved, or No Different or Worse — shown with the raw point difference. This is

computed and displayed even when the outcome is unfavourable.

VII. MANUAL PAR SCORING RUBRIC (REFERENCE)

For anyone extending the scoring UI or backend validation, the real rubric structure is:

Component	Weight	Sub-criteria
Upper Anterior Segment	×1	5 contact points: 3-2, 2-1, 1-1, 1-2, 2-3
Lower Anterior Segment	×1	Same 5 contact points
Buccal Occlusion (L/R)	×1 each	Antero-Posterior, Transverse (Crossbite), Vertical (Open Bite)
Overjet	×6	Type toggle (Positive / Reverse-Crossbite) + mm band
Overbite	×2	Type toggle (Overbite / Anterior Open Bite) + coverage band
Centreline	×4	Single deviation scale

VIII. ML ENGINE

Admin-controlled model lifecycle: current status, latest version, approved-dataset count, best accuracy, and total training runs are all shown live. A new training run takes a model version label and an epoch count (10-500) and runs only on approved training datasets; the previous model file is backed up automatically before each run, and rollback to an earlier version is supported.

Note: The UI's own dataset-confidence warning states 100 approved datasets as the recommended minimum for reliable predictions. Don't confuse this with any other confidence threshold you may find elsewhere in the code or docs — check the current source before citing a specific number in a report.

IX. TRAINING DATASET PIPELINE

Undergraduate submits an anonymised case label, a ground-truth PAR score (verified by their supervising orthodontist), optional source description, and three model files, assigned to a specific orthodontist reviewer. That orthodontist approves or rejects it. Only approved sets are eligible for the next training run. Admins have a read-only Submissions Overview across all users — approval/rejection authority stays with the assigned reviewer, not admins.

X. AUDIT LOGGING

Every significant action is logged with a timestamp, user, role, action type, entity, and structured detail. Known action types observed in practice:

- LOGIN
- FINALIZE CASE — detail includes stage and final parScore
- CALCULATE PAR — manual scoring save
- ML PREDICT LANDMARKS — detail includes points predicted and detector model version
- AUTO CALCULATE PAR — geometric score from confirmed landmarks, detail includes landmark count and weighted score
- ML PREDICTION — independent ML PAR prediction, detail includes predicted score and a confidence/experimental note

Use this taxonomy when adding new loggable actions — keep the ACTION name a short verb phrase and put the specifics in the structured detail field, matching the existing style.

XI. TESTING

123 backend automated tests (docs/TestingDocumentation.md, last re-verified 2026-09-24). 42 manual test cases (docs/ManualTestCases.md) — keep both current as behavior changes; the three-scoring-path distinction in §6.3 is exactly the kind of thing that's easy to test incorrectly if conflated.

XII. TROUBLESHOOTING

- Fresh clone can't run ./mvnw — needs a full JDK (javac), not just a JRE
- ML predictions look wildly different from manual/landmark scores — expected at low approved-dataset counts (see §8); not necessarily a bug
- See docs/LandmarkDetectorFixHistory.md for ML-service-specific coordinate/axis bugs already diagnosed

XIII. DEPLOYMENT

Currently local Docker Compose only. Document actual steps here if/when this moves to a hosting provider (CI/CD, production environment variables, TLS, etc.).